

IEE3584 - Comunicaciones inalámbricas

Tarea 12

Nicolás Seguel

Noviembre 2025

Problema 29

(1) Generación de símbolos QPSK

Se generaron símbolos QPSK independientes y equiprobables para dos antenas transmisoras, construyendo la matriz:

$$\mathbf{S} \in \mathbb{C}^{2 \times N_s}, \quad S_{ij} \in \left\{ \frac{1}{\sqrt{2}}(\pm 1 \pm j) \right\}.$$

Cada símbolo tiene energía unitaria, por lo que la energía total transmitida por instante se fija a $E_s = 2E_{\text{sym}}$. El código siguiente genera la matriz $\mathbf{S}$ y grafica las dos realizaciones de los símbolos generados para ambas antenas transmisoras.

```
# -----
# 1) Generar S (QPSK)
# -----
def gen_S_qpsk(seed: int, Ns: int = 1000, Es: float = 1.0) -> np.ndarray:
    """
    Genera matriz S de símbolos QPSK iid de forma (2, Ns).

    Parámetros
    -----
    seed : int
        Semilla para RNG (reproducible).
    Ns : int
        Número de columnas (número de bloques).

    Retorna
    -----
    S : ndarray (2, Ns)
        Matriz de símbolos complejos QPSK.
    """
    rng = np.random.default_rng(seed)
    # QPSK: puntos (+1 ± j)/sqrt(2) -> energía unitaria por símbolo
    bits_i = rng.integers(0, 2, size=(2, Ns))
    bits_q = rng.integers(0, 2, size=(2, Ns))
    # map: 0 -> +1, 1 -> -1
    a = 1 - 2*bits_i
```

```

b = 1 - 2*bits_q
S = (a + 1j*b) / np.sqrt(2) * Es    # energía por símbolo = 1
return S

```

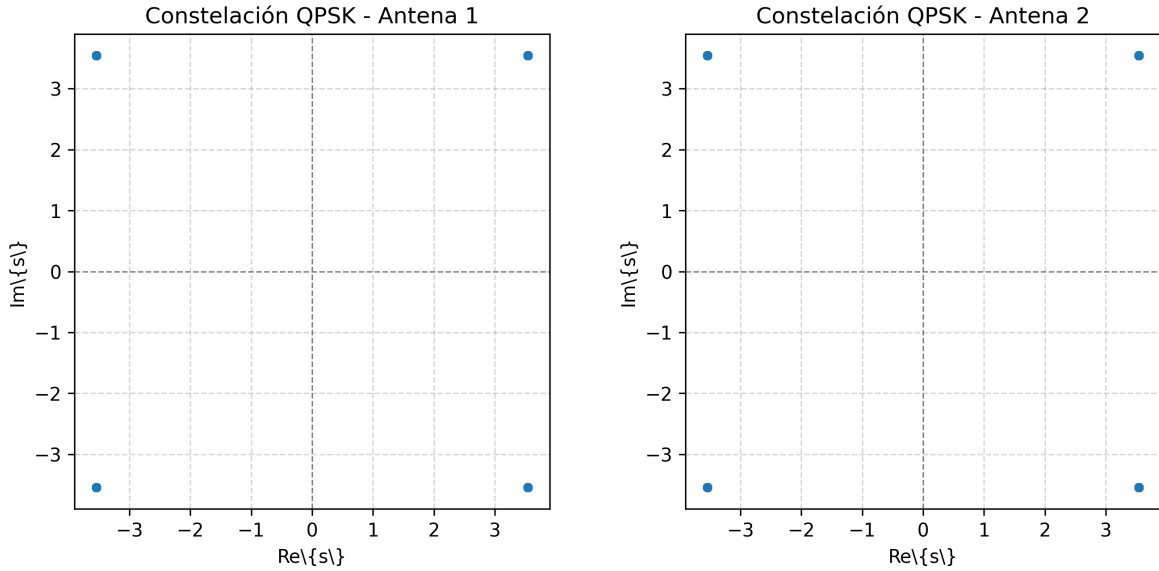


Figura 1: Constelaciones QPSK generadas para las dos antenas transmisoras.

(2) Codificación Alamouti

El esquema de Alamouti agrupa los símbolos s_1, s_2 en bloques de tiempo-espacio (STB):

$$\mathbf{X}_n = \begin{bmatrix} s_1 & -s_2^* \\ s_2 & s_1^* \end{bmatrix},$$

resultando en una matriz global $\mathbf{X}$ de dimensión $2 \times 2N_s$.

```

# -----
# 2) Codificador Alamouti
# -----
def alamouti_encode(S: np.ndarray)-> np.ndarray:
    """
    Codifica la matriz S (2 x Ns) usando el esquema Alamouti por bloque.
    Cada columna de S: [s1; s2] se transforma en un bloque 2x2:
        [ s1  -conj(s2) ]
        [ s2   conj(s1) ]
    Concatenando sobre Ns obtenemos X de tamaño (2, 2*Ns).

    Parámetros
    -----
    S : ndarray (2, Ns)

    Retorna

```

```

-----
X : ndarray (2, 2*Ns)
"""
if S.ndim != 2 or S.shape[0] != 2:
    raise ValueError("S must be shape (2, Ns)")
Nr, Ns = S.shape

s1 = S[0, :]      # fila 1
s2 = S[1, :]      # fila 2

# Matriz X de 2 x (2*Ns)
X = np.zeros((2, 2*Ns), dtype=complex)

# Columnas impares (en Python los índices comienzan en 0 → posiciones 0,2,4,...)
X[0, 0::2] = s1
X[1, 0::2] = s2

# Columnas pares (índices 1,3,5,...)
X[0, 1::2] = -np.conj(s2)
X[1, 1::2] = np.conj(s1)
return X

```

(3) Generación de la matriz de canal H

El canal se modela como Rayleigh plano, con coeficientes complejos iid:

$$h_{rt} \sim \mathcal{CN}(0, 1), \quad r, t = 1, 2.$$

El código usado fue el siguiente:

```

# -----
# 3) Generar H (2x2) Rayleigh CN(0,1)
# -----
def gen_H(seed: int):
    """
    Genera matriz H (2 x 2) con entradas CN(0,1):
        real ~ N(0, 1/2), imag ~ N(0, 1/2)
    Parámetros
    -----
    seed : int
        Semilla para reproducibilidad.

    Retorna
    -----
    H : ndarray (2,2) complejo
    """
    rng = np.random.default_rng(seed)
    H = (rng.normal(size=(2,2)) + 1j*rng.normal(size=(2,2))) / np.sqrt(2.0)
    return H

```

Los valores obtenidos para una realización fueron:

$$\mathbf{H} = \begin{bmatrix} h_{11} & h_{12} \\ h_{21} & h_{22} \end{bmatrix}$$

$$\begin{array}{ll} h_{11} = 0.63359593-1.84043293j & h_{12} = -1.50185841-0.13439643j \\ h_{21} = 1.15979924-0.46959669j & h_{22} = -0.0249661 -0.29323912j \end{array}$$

Cuadro 1: Realización de la matriz de canal $\mathbf{H}$.

(4) Generación del ruido y energía transmitida

El ruido térmico se genera como $\mathbf{N} \sim \mathcal{CN}(0, N_0)$, con $N_0 = 1$. Para una SNR objetivo $\bar{\gamma} = 7$ dB, la energía total por símbolo se fija a:

$$E_s = N_0 \cdot 10^{\bar{\gamma}/10}.$$

Energía total utilizada:

$$E_s = 25,118864315095784$$

(5) Transmisión por el canal

La transmisión simulada se obtiene aplicando:

$$\mathbf{Y} = \mathbf{H}\mathbf{X} + \mathbf{N},$$

donde $\mathbf{Y}$ es la señal recibida.

(6) Decodificación Alamouti

El decodificador aplica la combinación óptima:

$$\hat{s}_1 = \sum_m (h_{m1}^* y_m(1) + h_{m2} y_m^*(2)), \quad \hat{s}_2 = \sum_m (h_{m2}^* y_m(1) - h_{m1} y_m^*(2)),$$

normalizando por $g = \sum_m (|h_{m1}|^2 + |h_{m2}|^2)$.

```
# -----
# 6) Decodificador de Alamouti (multi-receive antennas)
# -----
def alamouti_decode(Y: np.ndarray, H: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
    """
    Decodifica bloques Alamouti contenidos en Y, asumiendo H constante en todo X.
    - Y : (Nr, NT) recibido donde Nr = 2, NT = 2*Ns
    - H : (Nr, 2) canal complejo (2x2)
    Devuelve:
    - S_hat : (2, Ns) estimaciones de símbolos (complejos)
    - g_per_block : (Ns,) ganancia g para cada bloque (g = sum_m |h_m1|^2 + |h_m2|^2)

    Notas: se sigue la combinación clásica:
    s1_hat = sum_m ( conj(h_m1) * y_m(2n) + h_m2 * conj(y_m(2n+1)) )
    s2_hat = sum_m ( conj(h_m2) * y_m(2n) - h_m1 * conj(y_m(2n+1)) )
```

```

y se normaliza entre cada bloque por g para obtener estimates s1_est = s1_hat / g.
"""
Nr, NT = Y.shape
if NT % 2 != 0:
    raise ValueError("NT debe ser par (2*Ns).")
Ns = NT // 2

# Extraer columnas pares/ímpares
# y[:, 2n] -> first timeslot, y[:, 2n+1] -> second timeslot
S_hat = np.zeros((2, Ns), dtype=complex)
g_per_block = np.zeros(Ns, dtype=float)

# Extraer h_{m1}, h_{m2}
# H shape (Nr, 2)
h1 = H[:, 0] # vector (Nr,)
h2 = H[:, 1] # vector (Nr,)

# Precalculer |h|^2 sum por bloque (consta por bloque)
# g = sum_m (|h_m1|^2 + |h_m2|^2)
g_const = np.sum(np.abs(h1)**2 + np.abs(h2)**2)

for n in range(Ns):
    y1 = Y[:, 2*n] # (Nr,)
    y2 = Y[:, 2*n + 1] # (Nr,)

    # combinaciones por bloque (ve colas de ruido)
    s1_hat = np.vdot(h1, y1) + np.vdot(h2.conj(), y2.conj())

    s1_hat = np.sum(np.conj(h1) * y1) + np.sum(h2 * np.conj(y2))

    s2_hat = np.sum(np.conj(h2) * y1) - np.sum(h1 * np.conj(y2))

    # Ganancia g (es la misma para todos los bloques si H fijo)
    g = g_const

    # Guardar
    S_hat[0, n] = s1_hat / (g + 1e-18)
    S_hat[1, n] = s2_hat / (g + 1e-18)
    g_per_block[n] = g

return S_hat, g_per_block

```

(7) Cálculo de SNR por símbolo

Cada bloque proporciona una ganancia g común a los dos símbolos. La SNR instantánea por símbolo se calcula como:

$$\gamma_i = \frac{E_{\text{sym}}}{N_0} g_i = \frac{E_s}{2N_0} g_i.$$

El vector resultante es $\boldsymbol{\gamma} = [\gamma_1, \dots, \gamma_{N_s}]$.

Promedio de SNR obtenido:

$$\bar{\gamma} = 19,33216001537366 \text{ (lineal)} \Rightarrow 10 \log_{10}(\bar{\gamma}) = 12,86 \text{ dB.}$$

(8) SNR media por realización

El promedio temporal de γ_i entrega la SNR media del canal actual:

$$\bar{\Gamma} = \frac{1}{N_s} \sum_{i=1}^{N_s} \gamma_i.$$

(9) Simulación Monte Carlo

Se repite la simulación sobre $N_H = 10,000$ realizaciones de canal, acumulando la SNR media para cada una.

(10) Resultados finales

El promedio global de las SNRs permite estimar la ganancia de arreglo:

$$G_{\text{arr}} = \frac{\mathbb{E}[\bar{\Gamma}]}{\bar{\gamma}}.$$

Resultado obtenido:

$$G_{\text{arr}} = 2,0175484299939486 \text{ (lineal)} \Rightarrow 10 \log_{10}(G_{\text{arr}}) = 3,0482396848788147 \text{ dB.}$$

Código completo

```
"""
Funciones para el Problema 29 (Tarea 12)
"""

import numpy as np
import matplotlib.pyplot as plt
import os

# -----
# 1) Generar S (QPSK)
# -----
def gen_S_qpsk(seed: int, Ns: int = 1000, Es: float = 1.0) -> np.ndarray:
    """
    Genera matriz S de símbolos QPSK iid de forma (2, Ns).

    Parámetros
    -----
    seed : int
        Semilla para RNG (reproducible).
    Ns : int
        Número de columnas (número de bloques).

    Retorna
    -----
    S : ndarray (2, Ns)
        Matriz de símbolos complejos QPSK.
    """
    rng = np.random.default_rng(seed)
    # QPSK: puntos (±1 ± j)/sqrt(2) -> energía unitaria por símbolo
    bits_i = rng.integers(0, 2, size=(2, Ns))
```

```

bits_q = rng.integers(0, 2, size=(2, Ns))
# map: 0 -> +1, 1 -> -1
a = 1 - 2*bits_i
b = 1 - 2*bits_q
S = (a + 1j*b) / np.sqrt(2) * Es # energía por símbolo = 1
return S

# -----
# 2) Codificador Alamouti
# -----
def alamouti_encode(S: np.ndarray)-> np.ndarray:
    """
    Codifica la matriz S (2 x Ns) usando el esquema Alamouti por bloque.
    Cada columna de S: [s1; s2] se transforma en un bloque 2x2:
        [ s1  -conj(s2) ]
        [ s2   conj(s1) ]
    Concatenando sobre Ns obtenemos X de tamaño (2, 2*Ns).

    Parámetros
    -----
    S : ndarray (2, Ns)

    Retorna
    -----
    X : ndarray (2, 2*Ns)
    """
    if S.ndim != 2 or S.shape[0] != 2:
        raise ValueError("S must be shape (2, Ns)")
    Nr, Ns = S.shape

    s1 = S[0, :] # fila 1
    s2 = S[1, :] # fila 2

    # Matriz X de 2 x (2*Ns)
    X = np.zeros((2, 2*Ns), dtype=complex)

    # Columnas impares (en Python los índices comienzan en 0 + posiciones 0,2,4,...)
    X[0, 0::2] = s1
    X[1, 0::2] = s2

    # Columnas pares (índices 1,3,5,...)
    X[0, 1::2] = -np.conj(s2)
    X[1, 1::2] = np.conj(s1)
    return X

# -----
# 3) Generar H (2x2) Rayleigh CN(0,1)
# -----
def gen_H(seed: int):
    """
    Genera matriz H (2 x 2) con entradas CN(0,1):
        real ~ N(0, 1/2), imag ~ N(0, 1/2)
    Parámetros
    -----
    seed : int
        Semilla para reproducibilidad.

    Retorna
    -----
    H : ndarray (2,2) complejo
    """
    rng = np.random.default_rng(seed)
    H = (rng.normal(size=(2,2)) + 1j*rng.normal(size=(2,2))) / np.sqrt(2.0)
    return H

# -----
# 4) Generar ruido N (2 x NT)

```

```

# -----
def gen_noise(seed: int, NO: float, NT: int) -> np.ndarray:
    """
    Genera ruido complejo blanco aditivo (2 x NT) con varianza compleja NO por
    muestra (es decir, var(real)=var(imag)=NO/2).

    Parámetros
    -----
    seed : int
    NO : float
        Potencia de ruido por muestra compleja.
    NT : int
        Número de columnas (tiempos) del bloque STB (NT = 2*Ns).

    Retorna
    -----
    N : ndarray (2, NT) complejo
    """
    rng = np.random.default_rng(seed)
    sigma = np.sqrt(NO/2.0)
    N = sigma * (rng.normal(size=(2, NT)) + 1j * rng.normal(size=(2, NT)))
    return N

# -----
# 5) Transmisión del STB X por el canal H y adición de ruido
# -----
def transmit_STB(H: np.ndarray, X: np.ndarray, N: np.ndarray) -> np.ndarray:
    """
    Transmite X por el canal H y añade ruido N:
    Y = H @ X + N

    Parámetros
    -----
    H : (2,2) ndarray complejo
    X : (2, NT) ndarray complejo (NT = 2*Ns)
    N : (2, NT) ndarray complejo

    Retorna
    -----
    Y : (2, NT) ndarray complejo
    """
    return H @ X + N

# -----
# 6) Decodificador de Alamouti (multi-receive antennas)
# -----
def alamouti_decode(Y: np.ndarray, H: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
    """
    Decodifica bloques Alamouti contenidos en Y, asumiendo H constante en todo X.
    - Y : (Nr, NT) recibido donde Nr = 2, NT = 2*Ns
    - H : (Nr, 2) canal complejo (2x2)
    Devuelve:
    - S_hat : (2, Ns) estimaciones de símbolos (complejos)
    - g_per_block : (Ns,) ganancia g para cada bloque (g = sum_m |h_m1|^2 + |h_m2|^2)

    Notas: se sigue la combinación clásica:
    s1_hat = sum_m ( conj(h_m1) * y_m(2n) + h_m2 * conj(y_m(2n+1)) )
    s2_hat = sum_m ( conj(h_m2) * y_m(2n) - h_m1 * conj(y_m(2n+1)) )
    y se normaliza entre cada bloque por g para obtener estimates s1_est = s1_hat / g.
    """
    Nr, NT = Y.shape
    if NT % 2 != 0:
        raise ValueError("NT debe ser par (2*Ns).")
    Ns = NT // 2

    # Extraer columnas pares/ímpares
    # y[:, 2n] -> first timeslot, y[:, 2n+1] -> second timeslot
    S_hat = np.zeros((2, Ns), dtype=complex)

```



```

g_per_block = np.zeros(Ns, dtype=float)

# Extraer h_{m1}, h_{m2}
# H shape (Nr, 2)
h1 = H[:, 0] # vector (Nr,)
h2 = H[:, 1] # vector (Nr,)

# Precalculer |h|^2 sum por bloque (consta por bloque)
# g = sum_m (|h_m1|^2 + |h_m2|^2)
g_const = np.sum(np.abs(h1)**2 + np.abs(h2)**2)

for n in range(Ns):
    y1 = Y[:, 2*n] # (Nr,)
    y2 = Y[:, 2*n + 1] # (Nr,)

    # combinaciones por bloque (ve colas de ruido)
    s1_hat = np.vdot(h1, y1) + np.vdot(h2.conj(), y2.conj() )

    s1_hat = np.sum(np.conj(h1) * y1) + np.sum(h2 * np.conj(y2))

    s2_hat = np.sum(np.conj(h2) * y1) - np.sum(h1 * np.conj(y2))

    # Ganancia g (es la misma para todos los bloques si H fijo)
    g = g_const

    # Guardar
    S_hat[0, n] = s1_hat / (g + 1e-18)
    S_hat[1, n] = s2_hat / (g + 1e-18)
    g_per_block[n] = g

return S_hat, g_per_block

# -----
# 7) Calcular la SNR de cada símbolo en la salida del decodificador
# -----
def snr_per_symbol_from_g(g_per_block: np.ndarray, Es: float, NO: float) -> np.ndarray:
    """
    Calcula la SNR por símbolo (lineal) a partir de g por bloque:
    gamma_sym = (E_sym / NO) * g
    donde E_sym = Es/2 (energía por símbolo por antena) en la convención usada.

    Parámetros
    -----
    g_per_block : (Ns,) ndarray
        g para cada bloque.
    Es : float
        Energía total por instante (suma sobre transmit antennas).
    NO : float
        Potencia de ruido por muestra compleja.

    Retorna
    -----
    gamma_sym : (Ns,) ndarray
        SNR por símbolo (lineal). Cada bloque tiene dos símbolos con mismo gamma.
    """
    E_sym = Es / 2.0
    gamma_sym = (E_sym / NO) * g_per_block
    # Cada bloque corresponde a DOS símbolos (s1 y s2) con la misma SNR:
    # Para conveniencia devolvemos vector con longitud Ns (una SNR por par),
    # quien necesite SNR por símbolo puede replicar cada valor dos veces.
    return gamma_sym

# -----
# 8) Promedio de la SNR por símbolos de la realización (sobre Ns pares)
# -----
def mean_snr_per_realization(gamma_sym: np.ndarray) -> float:
    """

```

```

Calcula la SNR promedio sobre todos los símbolos (a partir de gamma por bloque).
Para convertir a promedio por símbolo: replicar cada gamma_sym dos veces si hace falta,
pero la media de bloques es la misma que la media por símbolo.
"""
return float(np.mean(gamma_sym))

```

```

# -----
# 9 y 10) Loop Monte Carlo sobre NH canales: repetir pasos 1..8 y acumular estadísticas
# -----
def montecarlo_runs(NH: int,
                    Ns: int,
                    Es: float,
                    NO: float,
                    seed_base_symbols: int = 1000,
                    seed_base_channel: int = 2000,
                    seed_base_noise: int = 3000) -> dict:
    """
    Ejecuta NH realizaciones independientes del canal. Para cada i:
    - genera S con semilla seed_base_symbols + i
    - codifica Alamouti -> X
    - genera H con semilla seed_base_channel + i
    - genera ruido N con semilla seed_base_noise + i
    - transmite Y = H X + N
    - decodifica S_hat, obtiene g_per_block
    - calcula gamma_sym por bloque y su media sobre Ns
    Devuelve estadísticas:
    - mean_snr_list : list de NH valores (SNR media por realización, lineal)
    - overall_mean_snr : promedio sobre las NH realizaciones
    - array_gain_est : overall_mean_snr / gamma_bar (donde gamma_bar = Es/NO)
    - adicional: histogram data y vectores
    """

    mean_snr_list = np.zeros(NH, dtype=float)

    NT = 2 * Ns
    # barra_gamma de referencia (según convención usada antes)
    gamma_bar = Es / NO

    for i in range(NH):
        # semillas separadas
        s_seed = seed_base_symbols + i
        h_seed = seed_base_channel + i
        n_seed = seed_base_noise + i

        # 1) símbolos S
        S = gen_S_qpsk(seed=s_seed, Ns=Ns, Es=Es)          # (2, Ns)

        # 2) codificar Alamouti
        X = alamouti_encode(S)                             # (2, 2*Ns)

        # 3) generar canal
        H = gen_H(seed=h_seed)                             # (2,2)

        # 4) ruido
        N = gen_noise(seed=n_seed, NO=NO, NT=NT)          # (2, NT)

        # 5) transmitir
        Y = transmit_STB(H, X, N)                          # (2, NT)

        # 6) decodificar
        S_hat, g_per_block = alamouti_decode(Y, H)

        # 7) calcular SNR por bloque (par de símbolos)
        gamma_sym = snr_per_symbol_from_g(g_per_block, Es, NO) # length Ns

        # 8) promedio por realización
        mean_gamma = mean_snr_per_realization(gamma_sym)
        mean_snr_list[i] = mean_gamma

```

```

        # opcional: progreso
        if (i+1) % 1000 == 0:
            print(f"MonteCarlo: {i+1}/{NH} realizados ...")

    overall_mean_snr = float(np.mean(mean_snr_list))
    array_gain_est = overall_mean_snr / gamma_bar

    results = {
        "mean_snr_list": mean_snr_list,
        "overall_mean_snr": overall_mean_snr,
        "array_gain_est": array_gain_est,
        "gamma_bar": gamma_bar
    }
    return results

# =====
# Función auxiliar: scatter plot
# =====
def plot_constellation(S, title, filename):
    """
    Dibuja la constelación QPSK de una fila de S (Ns símbolos).
    """
    plt.figure(figsize=(4.5, 4.5))
    plt.plot(np.real(S), np.imag(S), 'o', markersize=4, alpha=0.7)
    plt.axhline(0, color='gray', linestyle='--', linewidth=0.8)
    plt.axvline(0, color='gray', linestyle='--', linewidth=0.8)
    plt.grid(True, linestyle='--', alpha=0.5)
    plt.title(title)
    plt.xlabel("Re\{s\}")
    plt.ylabel("Im\{s\}")
    plt.gca().set_aspect('equal', 'box')
    plt.tight_layout()
    plt.savefig(f"figs/{filename}", dpi=300)
    plt.close()

# =====
# (2) Graficar estructura Alamouti codificada
# =====
def plot_alamouti_structure(S):
    """
    Codifica S con Alamouti y muestra un bloque ilustrativo de los 4 primeros símbolos.
    """
    X = alamouti_encode(S)
    print(f"X generada: shape={X.shape} (bloques Alamouti)")

    # Seleccionar primer bloque (dos tiempos)
    s1 = S[0, 0]
    s2 = S[1, 0]
    block = np.array([[s1, -np.conj(s2)],
                      [s2, np.conj(s1)]])

    fig, ax = plt.subplots(figsize=(5, 3))
    im = ax.imshow(np.angle(block), cmap="twilight", vmin=-np.pi, vmax=np.pi)
    for i in range(2):
        for j in range(2):
            text = f"{block[i,j].real:+.2f}+{block[i,j].imag:+.2f}j"
            ax.text(j, i, text, ha='center', va='center', color='w', fontsize=8)
    plt.xticks([0, 1], [r"$1$", r"$2$"])
    plt.yticks([0, 1], [r"Antena 1", r"Antena 2"])
    plt.title("Estructura del bloque Alamouti")
    plt.colorbar(im, label="Fase [rad]")
    plt.tight_layout()
    plt.savefig("figs/problema29_alamouti_block.png", dpi=300)
    plt.close()

    return X

# -----

```

```

# Ejemplo de uso / calibración
# -----
def example_usage():
    """
    Ejemplo de cómo invocar el MonteCarlo.
    ADVERTENCIA: NH=10000 puede tardar; prueba con NH=200 primero.
    """
    NH = 10000
    Ns = 1000
    # calibración: gamma=7 dB, NO=1 -> Es = gamma_lin * NO
    gamma_db = 7.0
    gamma_lin = 10**(gamma_db/10.0)
    NO = 1.0
    Es = gamma_lin * NO

    seed_symbols = 123
    seed_channel = 456
    seed_noise = 789

    os.makedirs("figs", exist_ok=True)

    # 1) generar S
    S = gen_S_qpsk(seed=seed_symbols, Ns=Ns, Es=Es)
    print("S shape:", S.shape, "E[|s|^2] per symbol (each antenna):", np.mean(np.abs(S)**2))
    plot_constellation(S[0, :], "Constelación QPSK - Antena 1", "problema29_constellation_antenna1.png")
    plot_constellation(S[1, :], "Constelación QPSK - Antena 2", "problema29_constellation_antenna2.png")

    # 2) codificar Alamouti
    X = alamouti_encode(S)
    plot_alamouti_structure(S)

    # 3) generar H (2x2)
    H = gen_H(seed=seed_channel)
    print("H:\n", H)

    # 4) generar ruido N (2 x NT)
    NT = X.shape[1]
    N = gen_noise(seed=seed_noise, NO=NO, NT=NT)
    print("N shape:", N.shape)

    # 5) transmitir
    Y = transmit_STB(H, X, N)
    print("Y shape:", Y.shape)

    # 6) decodificar
    S_hat = alamouti_decode(Y, H)
    print("S_hat shape:", S_hat[0].shape)

    # 7 y 8) calcular SNR por bloque
    g_per_block = S_hat[1]
    gamma_sym = snr_per_symbol_from_g(g_per_block, Es, NO)
    print("gamma_sym shape:", gamma_sym.shape)
    mean_gamma = mean_snr_per_realization(gamma_sym)
    print("Mean SNR per realization (lineal):", mean_gamma)

    # 9 y 10) MonteCarlo
    res = montecarlo_runs(NH=NH,
                          Ns=Ns,
                          Es=Es,
                          NO=NO,
                          seed_base_symbols=1000,
                          seed_base_channel=2000,
                          seed_base_noise=3000)
    print("Overall mean SNR (lineal):", res["overall_mean_snr"])
    print("Gamma_bar (ref):", res["gamma_bar"])
    print("Estimated array gain (linear):", res["array_gain_est"])
    print("Estimated array gain (dB):", 10*np.log10(res["array_gain_est"]))

```

```
if __name__ == "__main__":  
    example_usage()
```